

NHH



Norwegian School of Economics
Bergen, November 2023

BAN401

Applied Programming and Data Analysis for Business

Assignment 2

Candidate numbers: 65, 146 and 370

Contents

Problem 1	3
Python code	3
Explanation of your solution	4
Screenshots of the results from VSCode	5
Problem 2	6
Python code	6
Explanation of your solution	7
Screenshots of the results from VSCode	8
Problem 3	9
The use of Python in the healthcare sector	9
Introduction	9
Analysis	9
Data collection	9
Data preprocessing	10
Data analysis	11
Prediction	11
Classification	12
Decision making	13
Visualization	13
Case studies	13
Cancer study screening – Amazon Comprehend Medical	13
Treatment adherence monitoring - AiCure	14
Operational efficiency improvements - Qventus	14
Cancer diagnostics – Paige	15
Conclusion	15
Problem 4	16
Select your business scenario, and define the problem	16
R code	16
Explanation of your solution	16
Screenshots of the results from RStudio	17
Problem 5	18
R code	18
Explanation of your solution	18
Screenshots of the results from the RStudio	19
Problem 6	20
Database Idea	20
ER-Model	20

Database Creation	22
SQL Code Documentation	22
Inserting data to the tables	25
Description for each relationship (between tables)	25
Describing each table	26

Problem 1

Python code

```

1 # problem 1
2
3 # Starting by defining the text we are going to use as the input_text
4 input_text = (
5     "Hey, how's everyone doing? Pretty good! Just got back from a weekend hiking "
6     "trip with breathtaking views. I wish I could've joined. Any plans for "
7     "summer vacation? Well, I was thinking of a beach getaway with sun, sea, and "
8     "relaxation. Hei can you turn on the lights? Sounds heavenly! Count me in. I "
9     "might join too, but first, I need to finish this coding project I've been "
10    "working on. Speaking of work, have you seen the latest company newsletter? "
11    "Hei can you close the curtains? Yeah, I skimmed through it. They're "
12    "introducing a new benefits package, which is cool. We could use some extra "
13    "perks. Agreed, but I hope they improve the coffee machine too! Haha, "
14    "priorities! Speaking of coffee, do you guys prefer iced or hot coffee? Iced "
15    "coffee, especially in the summer, is my go-to. Hei can you play some music! "
16    "I'm all about hot coffee, no matter the season. I like to mix it up – iced "
17    "in the morning, hot in the afternoon. Variety keeps it interesting! Well, "
18    "whatever keeps us caffeinated, right? Absolutely! Speaking of caffeine, "
19    "anyone up for a coffee run now? Sounds like a plan! Count me in. Let's do "
20    "it! hei assistant turn off the TV!")
21
22 trigger_word = "hei" # initialize the trigger word
23
24 # Defining a function to return the position of the trigger word.
25 # The function takes two arguments:
26 # 1.st argument: the text to be analysed
27 # 2.nd argument: the trigger word
28 def trigger_word_positions(text, trigger_word):
29     text_lower = text.lower() # making text case-insensitive
30     words = text_lower.split() # splitting the text into words
31     # iterating over words to find the positions
32     trigger_word_positions_in_words = []
33     # checking for match or surrounding punctuation
34     for i, word in enumerate(words):
35         if word.strip('.,!?') == trigger_word:
36             trigger_word_positions_in_words.append(i)
37     return trigger_word_positions_in_words # returning position
38 n = trigger_word_positions(input_text, trigger_word=trigger_word)
39
40 # Defining a new function to return extracted sentences
41 def extract_complete_sentences(text, positions):
42     text_lower = text.lower() # normalising text to solve case-insensitivity
43     words = text_lower.split() # splitting the input text into words
44     extracted_sentences = [] # extract sentence following trigger word
45     for position in positions:
46         sentence_words = [] # create sentences from trigger word onwards
47         for word in words[position + 1:]: # stop when reaching end
48             if word.endswith('.') or word.endswith('!?') or word.endswith('?!'):

```

```

49         sentence_words.append(word.strip('.,!?'))
50         break
51     else:
52         sentence_words.append(word.strip('.,!?'))
53         extracted_sentences.append(' '.join(sentence_words))
54     return extracted_sentences          # return the sentence
55
56 # finding the positions of trigger words
57 test_positions = trigger_word_positions(input_text, trigger_word)
58
59 # extracting sentences following the trigger word position
60 extracted_sentences = extract_complete_sentences(
61     text = input_text,
62     positions = test_positions)
63
64 # displaying the result
65 for position, sentence in zip(test_positions, extracted_sentences):
66     print(f"Trigger word 'hei' found at position {position}:")
67     print(sentence) # printing the extracted sentences
68     print("")      # creating some space

```

Explanation of your solution

In our python code we did not import any modules or packages. We start by defining the `input_text`, from which we will investigate. Thereafter we set the `trigger_word`, which in this case is "hei".

The first function `find_trigger_word_positions` take `trigger_word` and `text` as arguments. Thereafter the function makes the input text lowercase, to make sure the searching algorithm is case-insensitive. Furthermore, the input text is split into words, using `normalized_text.split()`. The function thereafter uses a for-loop searching for words in the text equal to the trigger word, and returning its position.

The second function `extract_sentences` takes arguments `text` and `positions`, where its function is to extract the sentences starting with the trigger word. First we normalise the text to lowercase making it case-insensitive. Thereafter input text is split into words as before. Starting from the trigger words in the sentence, we loop over each word of the sentence starting with the `trigger_word`, ending at a ".". If the next word is not the end of the sentence, that word is joined with the current sentence, which is returned as the `extracted_sentences`.

Using these functions together we can first identify the position of `trigger_word` using the `find_trigger_word_positions`, and thereafter extract the sentence starting with that word using the `extract_sentences` function.

Screenshots of the results from VSCode

Trigger word 'hei' found at position 40:
can you turn on the lights

Trigger word 'hei' found at position 79:
can you close the curtains

Trigger word 'hei' found at position 137:
can you play some music

Trigger word 'hei' found at position 198:
assistant turn off the tv

Problem 2

Python code

```

1 # We use the combinations function from itertools
2 from itertools import combinations
3 max_weight = 300 # Define the maximum weight
4
5 # Define all items and , weights and values
6 items = [
7     ("hiking boots", 36, 90),
8     ("BAN401 printed slides", 60, 110),
9     ("water bottle", 20, 50),
10    ("energy bars", 30, 40),
11    ("first aid kit", 40, 60),
12    ("flashlight", 10, 30),
13    ("pocketknife", 15, 45),
14    ("sunscreen", 25, 70),
15    ("sunglasses", 7, 20),
16    ("rain jacket", 45, 80),
17    ("hat", 12, 25),
18    ("camera", 50, 100)
19 ]
20 # Define a function to calculate weight and value
21 def calculate_total_weight_and_value(combination):
22     total_weight = sum(item[1] for item in combination)
23     total_value = sum(item[2] for item in combination)
24     return total_weight, total_value
25 # Defining the values to add later in the iteration
26 best_combination = None
27 best_value = 0 # Will be the best value combination
28 best_weight = 0 # Will be the best weight combination
29 # Iterating over all possible item combinations
30 for i in range(1, len(items) + 1):
31     for combination in combinations(items, i):
32         total_weight, total_value = calculate_total_weight_and_value(
33             ↪ combination)
34         if total_weight <= max_weight and total_value > best_value:
35             # updating the values if the total weight is smaller
36             # and the best value is better
37             best_combination = combination
38             best_value = total_value
39             best_weight = total_weight
40 # Output
41 optimal_items = "\n".join(item[0] for item in best_combination)
42 output = f"\nThe optimal combination of items:\n{optimal_items}\n\n"
43 output += f"The total value is:\n{best_value}\n\n"
44 output += f"and the total weight is:\n{best_weight}\n\n"
45 print(output)

```

Explanation of your solution

The problem presented is a classic example of the Knapsack problem, a fundamental question in combinatorial optimization. The objective is to select a subset of items with given weights and values to maximize the total value without exceeding the maximum weight capacity of the knapsack, in this case, the backpack with a 3 kg limit.

Our solution to this problem employs a brute-force approach using the power of combinatorial mathematics. Here's a step-by-step breakdown of the solution:

Item Representation: Each item is represented as a tuple containing the item's name, weight, and value. All items are stored in a list. **Combination Generation:** Utilizing the `combinations` function from Python's `itertools` module, we generate all possible combinations of items. This ensures that every potential combination of items is considered. **Evaluation of Combinations:** Each combination is evaluated by calculating its total weight and total value. This is done using a helper function `calculate_total_weight_and_value`, which sums up the weights and values of all items in a combination. **Selection Criteria:** The combination that yields the maximum total value without exceeding the weight limit of 3000 g (3 kg) is considered optimal. To find this, I iterate through all combinations, keeping track of the best combination found so far based on the total value. If a combination exceeds the weight limit, it is disregarded. **Result Presentation:** Once the optimal combination is found, the solution is presented in a reader-friendly format. The output includes the names of the items in the best combination, the total value of these items, and their total weight.

This brute-force approach guarantees finding the optimal solution, as it explores every possible combination of items. However, it's worth noting that the complexity of this solution is exponential with respect to the number of items, which can make it computationally expensive for a large number of items. For a relatively small set of items, as in this problem, this approach is both feasible and effective.

In conclusion, the solution leverages the power of combinatorial mathematics to explore all possible combinations of items and applies a simple yet effective evaluation criterion to identify the combination that maximizes value while adhering to the weight constraint.

Screenshots of the results from VSCode

The optimal combination of items:

hiking boots

BAN401 printed slides

water bottle

energy bars

flashlight

pocketknife

sunscreen

sunglasses

rain jacket

camera

The total value is:

635

and the total weight is:

298

Problem 3

The use of Python in the healthcare sector

Introduction

Among many other fields about to be reshaped by digital transformation lies the healthcare industry. The more data shifts from a paper to a digital format, the more insights can be gathered, leading to optimizations, predictions, and efficiency improvements. Besides, such improvements will not just be a positive externality of technological advancements in the world, they will be a necessity. Research by representatives from Accenture highlights how in the US, the aging of the population will strain the healthcare sector from both the demand and supply side (Shah et al., 2023). That is, a higher number of elderly people implies the need for more care, while increasingly more healthcare workers retire, and the education system fails to prepare enough qualified young specialists. This trend of aging population is similar in the rest of the developed world.

This essay aims to explore some of the possible ways in which Python can be utilized in the healthcare sector. We start by using the so-called data life cycle approach, identifying Python’s potential in data collection, preprocessing, analysis, and visualization separately. While we already offer some use cases in these steps, we further extend the analysis with four additional real-life use cases of Python.

Analysis

There are multiple frameworks that examine the progression of a data project, for example, “the data life cycle” approach lists eight steps, where the first one, data generation, happens even without the analyst’s intervention (Stobierski, 2021). We skip the data generation step and elaborate on Python’s usability in four other steps included in the framework (in the following order) – collection, preprocessing, analysis and visualization.

Data collection

Unsurprisingly, the healthcare industry has been shifting towards the digitization of patient data, either due to efficiency concerns or demands by law. For example, in the United States in 2009, less than 10 percent of the hospitals utilized electronic health records, while by 2014, this share of “digital healthcare institutions” had reached 97 percent (Bronsoler et al., 2022). These electronic health records mean that various results of one’s past medical procedures and examinations are available in a digital and shareable format (Bronsoler et al., 2022). This information can be imported into Python, which can be used as a foundation for the next steps in the data analysis (Accelebrate, n.d.). The most exciting prospect of Python within data collection is its ability to broaden the collection of a person’s health data points.

Sticking to the hospital environment, Python could leverage natural language processing to extract comprehensible data from various healthcare documents, including audio recordings or handwritten physicians’ notes (Asenavi, 2022). This leads to a more usable dataset, where

main keywords related to potential diagnoses can easily be identified without requiring a doctor to manually read the scanned notes of their colleagues.

There is an opportunity to extend the data collection beyond the hospital visits. A group of authors explore the promising angle of opportunities in data collection using smart mobile devices and wearables (Karthan et al., 2022). Their main motivation is the efficiency gain of utilizing mobile phones' sensors to gather the data required in medical studies and trials, replacing the slow manual collection process. Currently, there are multiple obstructions limiting the use of these phone sensors for data collection, including incompatibility across mobile phone operating systems, proprietary data formats and lack of tools' flexibility. While the new framework they developed to overcome these issues does not use Python, it could still have a part in the data collection from the smart devices, or at least come in handy at later stages of the data analysis. There is already a Python package called "wearablecompute", designed to transform data from mobile health apps and wearables "into digital biomarkers that act as indicators of health outcomes and can be used to diagnose, monitor, and manage chronic diseases and conditions" (Bent, 2020).

While useful for medical studies, a smooth and frequent health data integration from devices most already own could offer benefits to regular patients, too. The more data points there are, the better the opportunity for doctors to observe when a patient has deviated from their normal trends and cross-check possible issues with the person's medical record. It would be especially useful when there are suspicions of an illness, but people are allowed to go home whilst still required to watch out for certain symptoms that the phone or wearable sensors could measure. Python might then play a significant role in monitoring the data stream from the person's mobile devices and raising alarms when needed.

An opposite force for the data collection improvements – data privacy concerns. In the 2000s, state data privacy regulations were shown to stall the adoption of electronic health records in hospitals (Miller & Tucker, 2009). Today, the unease has shifted towards the use of external partners for the implementation of AI solutions in health companies and weak privacy sheltering in some of these partnerships, potentially risking unauthorized access to sensitive information (Murdoch, 2021). Yet, research exists on addressing this problem, describing how Python could be used in developing privacy-enhancing technologies, for example, using data encryption, so only authorized users could decrypt the otherwise unreadable health data (Jordan et al., 2022).

Data preprocessing

Data preprocessing is an unavoidable step ahead of analysis and visualization. Some of the required actions here are in line with what one could expect from any industry's data. Accelebrate (n.d.) offers a healthcare-centered Python course, where Pandas and NumPy functions are used to merge and clean the data, explore possibilities of imputation for the missing data, remove or fix errors and duplicates, etc.

The main healthcare data preprocessing consideration, however, is the question of how to turn unstructured data into structured. Hospitals generate massive amounts of data, but it is estimated that about 80% of it is unstructured (Eastwood, 2023). Plenty of detailed data ends up getting attached to a patient's health record as an unstructured summary. Hence, the idealized comprehensive digital overview of the patient falls short of expectations due to

the failure to structure the inputs.

The huge potential in solving the healthcare data structure issue has been recognized by Amazon Web Services (n.d.-a). They offer a service called “HealthLake”, which enables the customers (healthcare organizations) to plug in miscellaneous patient data, from reports and records to notes and insurance claims into their system, regardless of whether the data is structured or not. HealthLake applies relevant machine learning models to structure the data, which is then used to create easily searchable, analyzable structure data, usable for capturing trends and creating predictions. Given Python’s compatibility with machine learning and natural language processing, this illustrates an approach that could potentially have been achieved using Python instead. Alternatively, Amazon Web Services offers software development kits that facilitate HealthLake data accessibility through Python, too (Amazon Web Services, n.d.-b). To sum up, the main gain Python could offer in data preprocessing could be in structuring the unstructured data, allowing it to tap into valuable information that has often been overlooked.

Data analysis

Intuitively, the data analysis part is the one where Python can offer the widest array of improvements. A literature review by Salazar-Reyna et al. (2022) looking into data science, analytics and machine learning scientific papers aimed at improving the healthcare sector found “prediction” to be the most frequent application objective in the publications, followed by classification, decision making and data mining. Since data mining has been covered in the data collection section, we will offer possible Python uses in the other three most frequent directions.

Prediction

Firstly, predictive modeling. Theoretically, the opportunities here are limited only by analysts’ creativity. The popularity of these predictive models in healthcare is indicated by PyHealth Python library (Zhao et al., 2021). It encompasses a preprocessing model that can turn various healthcare datasets into formats usable by machine learning models. The predictive and evaluation models included in the library make it easy to run 30 different machine-learning models and gauge the model fit. Expecting even more research in this area, authors have created this library to raise reproducibility and comparability in the field.

Coming to specific use cases, Kumar et al. (2022) provide an extensive systematic literature review looking at what prediction models have been employed in academic research to predict individuals’ susceptibility of developing a specific disease. Authors offer statistics for what techniques have been used, the accuracy achieved and types of diseases the other researchers have tried to detect. Neural networks and tree methods seem to be the most popular, while Alzheimer’s, diabetes, liver, and heart issues are the most frequent diseases researchers have tried to diagnose. So, Python could be applied to proactively encourage screening a person for a particular disease even before they show up at the doctor’s office with symptoms. Countless other relevant predictive analyses are possible, like readmission risk, length of stay, staff demand, even equipment maintenance.

There is, however, a potential risk of predictions (or classifications, too) that cannot be overlooked. In the 1990s, a research project was launched in the US with the aim of healthcare cost reduction through a reduction in hospital admissions by more precise recognition of who requires hospitalization (Caruana et al., 2015). As a part of this study, the performance of multiple models predicting the probability of death for patients who show up at the hospital with pneumonia was compared. A high probability would indicate the need for hospitalization, while low-risk patients would get the treatment but not stay overnight. While multitask neural nets showed the highest accuracy, researchers decided against using them in practice due to a lack of “intelligibility”, that is, interpretability by humans. A related rules-based model had picked up a counter-intuitive rule that asthmatic pneumonia patients have a smaller death risk compared to the general population; a fact that mirrored the real-world data, where asthmatics were almost immediately taken to the intensive care unit and thus, had a lower death rate. Since these neural nets were not intelligible models, researchers could not know what other faulty bits of logic the models had developed from the training data. This should act as a cautionary tale – the pursuit of maximum predictive accuracy by using unintelligible models could lead to perverse outcomes.

Classification

Secondly, classification. Mechanically, classification tasks are rather similar to prediction problems. Even though the classification task’s output is assignment to a certain group, while a prediction task’s output is a probability (continuous variable), in their essence, both could be designed to evaluate the correlation between a patient and the disease. Therefore, PyHealth library’s tools can be used for classification problems in healthcare, too.

In its most frequent use, classification problems, instead of predicting the probability of just one particular disease, focus on determining a diagnosis, choosing among one of the possible options. Keniya et al.(2020) is just one group of researchers that have shown how, using symptom and demographic data, machine learning models can classify patients by their diseases with a high accuracy. Similarly, these models can be extended to image classification, spotting abnormalities in magnetic resonance imaging or X-rays. While humans might be skeptical of algorithms determining their diagnoses and, thus, expect doctors to make the final call, such classification models run in Python could be valuable assistants to healthcare practitioners, especially as AI becomes more sophisticated.

The classification tasks do not necessarily have to assign an exact diagnosis either. Some experts expect a rise of conversational AI, which would note person’s symptoms, go through their health records and then assign patients to one of the two groups: emergency versus primary care (Horowitz, 2022). Linking this to the growing field of telehealth, the conclusion might even be that a remote consultation would suffice. Either way, the hope is that non-urgent patient case would no longer block the access to help for those in immediate need. Moreover, these AI solutions will be easily able to remind people how to prepare for their hospital visit, whether they should skip breakfast, what they should bring with them, etc.

Decision making

Finally, data analysis with the aim of improving decision-making. Prediction and classification tasks can be a part of the improved decision-making pursuit, so again, there is not a very distinct separation here. However, since a clear decision is your expected end goal here, the analysis is more on the applied side. The machine learning models built in Python would then be run to, for example, personalize patient care (University of Pittsburgh, 2021). Based on the person's characteristics and observations from the field, one could determine what medicine should work the best and at what dosage, conditional on interactions with other drugs. Data analysis could also aid in determining whether a person is not too fragile for surgery, what alternative treatments could work, when someone is fit enough to leave the hospital, etc. One could also forecast, for instance, the intensity of the flu season, to figure out the staffing requirements ahead of time, raising operational efficiency.

Visualization

Data visualization is a step that you cannot separate from the previous ones. That is, the visualizations will reflect the available data and the analysis outcomes. Mohanty et al. (2021) demonstrate some examples of using Python libraries Matplotlib and Seaborn to generate basic graphical representations of medical data.

Moving on to more complex tools, Python's Open CV library opens new possibilities in medical picture processing (Widodo et al., 2020). In an overlap between visualization and preprocessing, Open CV library aids with medical image enhancement, noise reduction and segmentation. This enhanced visual data can then be used by machine learning algorithms to spot abnormalities or recognize diseases. Or it could just make the doctor's job easier as the critical parts in the medical images become more accentuated.

Another complex visualization use case is PyMOL (Yuan et al., 2017). In brief, faulty proteins or DNA are the sources of many illnesses. To prevent the negative consequences of these biological malfunctions, scientists must locate the target location for a drug molecule to bind, test a plethora of drug molecules as to whether they can bind and achieve the intended benefit. The Python-based PyMOL can create 3D representations of the biological target structures and simulate the interactions, immensely assisting in this time-consuming and difficult treatment creation process.

Case studies

In the previous sections, we have primarily focused on Python's use in healthcare from either a more academic perspective (meaning forwards looking or theoretical), or a hypothetical perspective. Most of those use cases were specific to a certain step in the data cycle. Now we extend the analysis with some examples from the real-world, which usually cover many steps of the data cycle.

Cancer study screening – Amazon Comprehend Medical

Given what is on the line, clinical studies are subject to strict regulations. To test the effectiveness of a very specific treatment, organizations will have a limited population of

eligible patients narrowed down by strict inclusion criteria, implying the necessity to go through the whole medical record of an individual to see whether they could be included in the clinical trial (Horowitz, 2022). The Fred Hutchinson Cancer Center, with the assistance of Amazon Comprehend Medical, employed natural language processing to exponentially increase the pace at which physicians were able to audit the patients' medical history and select the suitable individuals for a clinical trial. Although they do not explicitly mention the use of Python, his programming language could have gotten this task done.

Treatment adherence monitoring - AiCure

Continuing the topic of clinical trials, researchers might not be able to continuously monitor the participants. In a hypothetical scenario where participants are asked to take a certain dose of a new drug X times per day at home, the study authors might have to trust that people will follow the set guidelines. If they do not, however, the trials data could be distorted. AiCure is a healthcare technology company that uses AI to check whether the study participants follow the required dosage and frequency (AiCure, n.d.-a). Trial participants are prompted according to schedule to identify themselves on the phone or computer camera, show the medicine to be consumed and keep recording while, for example, swallowing the pill. That way researchers can monitor the adherence and reward the person each time they comply.

AiCure is not limited to clinical trials, though. It can be used by regular doctors to ensure their patients are adhering to the treatment schedule, especially in cases of diseases that affect devotion – depression, schizophrenia, and Parkinson's among others (AiCure, n.d.-b). During the aforementioned videos and extra prompts, like showing the patient a picture, asking to describe it and explain how it makes them feel, AiCure picks up slight cues in changes speech and movement patterns that signal changes in the patient's condition.

Given how AiCure team has published OpenDBM package in Python that “allows for calculation of digital biomarkers of health and functioning from video or audio of an individual's behaviour” and also frequently mentions Python requirement in their job advertisements, it is safe to assume Python is at the heart of their solutions (Florice et al., 2022).

Operational efficiency improvements - Qventus

Qventus is a US based company leveraging machine learning, artificial intelligence, and other data-related approaches to boost hospitals' operational efficiency (Qventus, n.d.-a). As the company reveals in a third-party performance report, the two development platforms used in Qventus' solutions are Python and JavaScript (Oscarson et al., 2023). The company splits their offering primarily into two parts, perioperative and inpatient.

The perioperative part can be summarized as AI-driven operating room scheduling for utilization maximization (Qventus, n.d.-b). Machine learning algorithms forecast which of the scheduled operations are likely to end early. Usually, the surgeons would announce this potentially unused time at the last minute, when it is already too late to assemble a team that could use the vacant operating room. The scheduling software proactively tries to nudge the surgeon to confirm they are indeed likely to finish early, so the algorithm can find another surgeon who could now use the newly opened time slot.

The inpatient solution is focused on minimizing the excess days of patient stays at the

hospital (Qventus, n.d.-c). The most illustrative case – patient has received their medical treatment, everything seems to be all right and they could get discharged today, except that “there is this final screening that must be done, but not today, since all the timeslots are taken”. Qventus is focusing on mastering the care scheduling to prevent these discharge barriers from appearing, resulting in shorter hospital stays, opening the beds for new patients.

Cancer diagnostics – Paige

Finally, Paige is a US based company behind the world’s first FDA-approved AI tool in pathology (Paige, n.d.-b). In brief, Paige has amassed “the world’s largest digital image library of cancer cases and associated genomic data from diverse sources”, which has been used in training their AI models in recognizing, quantifying, and highlighting cancer in tissue biopsies (Paige, n.d.-a). The software can look through the images, point out the suspicious spots and sort the patients in descending order by their likelihood of having cancer. Pathologists then manually review the cases, but already in a prioritized order. Independent studies have found Paige can provide up to 22% cut in the required time for pathologists to review one’s screening results, shorten time to receive the diagnosis by up to two-thirds and decrease diagnostic error by an impressive 70

Paige does not specifically list the programming languages they use. However, again, we have mentioned Python’s capabilities in similar image classification situations, so Python could have been used in this case even if it is not.

Conclusion

In this essay, we have looked at multiple directions in which data analysis in Python can provide value to the healthcare sector. Some of these applications so far seem to be stuck in the academic literature, some are seeing the first steps of implementation and are anticipated to materialize in the upcoming years. Meanwhile, approaches like classification and predictive analytics have almost endless opportunities in the field. Healthcare is headed towards a much more values-based approach, where Python will contribute to personalizing the care for everyone. A future of proactive instead of reactive approach, identifying diseases before it is too late, saving countless lives.¹.

¹For better readability of the report, the references used in this essay are added after Problem 6

Problem 4

Select your business scenario, and define the problem

For this problem we choose the business scenario of compounded interest. Imagine we are a financial institution wanting to plan saving money for the customer. For the input we will use the initial amount of money in the bank P , the monthly deposit C , the risk free rate r , and the number of years n .

R code

```

1  # recursive function
2  recursive <- function(P, C, r, n) {
3    if (n == 0) {
4      return(P)
5    } else {
6      last_year = recursive(P, C, r, n - 1)
7      return(last_year * (1 + r) + C * 12)
8    }
9  }
10
11 # iterative function using for-loop
12 iterative <- function(P, C, r, n) {
13   for (i in 1:n) {
14     P <- P * (1 + r) + C * 12
15   }
16   return(P)
17 }
18
19 # tuning parameters randomly
20 P = runif(1,100,10000) # initial investment
21 C = runif(1,100,500)    # monthly contribution
22 r = runif(1,0.001,1)    # risk free rate
23 n = sample(5:40,1)      # years until retirement
24
25 # printing the values of P, C, r, and n
26 c("P"=P,"C"=C,"r"=r,"n"=n)
27
28 # outputting recursive and iterative solutions
29 recursive(P,C,r,n)
30 iterative(P,C,r,n)

```

Explanation of your solution

Recursive solution: This solution shows clearly how the current periods balance is dependant on the previous period. As such, the recursive R function has high readability. It is clear that the function returns `last_year` times the risk free rate per year $(1+r)$ adding on the monthly deposits $C*12$. *Iterative solution:* This is somewhat more difficult to interpret. We

do know that the counter in the for loop goes from $1:n$ for every i , where the balance P will be $P*(1+r)+C*12$.

Screenshots of the results from RStudio

If we tune the parameters randomly, and output them, we see that for any combination of $c(P,C,r,n)$, it has the same output for the recursive and iterative solutions.

```
> c("P"=P, "C"=C, "r"=r, "n"=n)
      P          C          r          n
5786.1717630 244.1879948 0.7880033 34.0000000
> # outputing recursive and iterative solutions
> recursive(P,C,r,n)
[1] 3.617961e+12
> iterative(P,C,r,n)
[1] 3.617961e+12
```

Problem 5

R code

```

1 # finding open doors at final state
2 open_doors <- c()
3
4 # iterating through doors 1 to 930
5 for (door in 1:930) {
6   # check if door number is perfect square
7   if (sqrt(door) == floor(sqrt(door))) {
8     # add door number to open door list
9     open_doors <- c(open_doors, door)
10    # break loop when 30 doors found
11    if (length(open_doors) == 30) {
12      break
13    }
14  }
15 }
16
17 # printing the first 30 open doors
18 print(open_doors)
19
20 # finding closed doors at final state
21 closed_doors <- c()
22
23 # iterating through doors 1 to 930
24 for (door in 1:930) {
25   # check if door number not perfect square
26   if (sqrt(door) != floor(sqrt(door))) {
27     # add door number to closed door list
28     closed_doors <- c(closed_doors, door)
29     # break loop when 30 doors found
30     if (length(closed_doors) == 30) {
31       break
32     }
33   }
34 }
35
36 # printing the first 30 closed doors
37 print(closed_doors)

```

Explanation of your solution

The concept behind our designed solution: We do know that the doors starts in a closed state, such that for every door, it will stay open/closed at the final state if it has been visited by an even/odd number of individuals respectively. Furthermore, we do know that doors are visited by a limited number of individuals, i.e. door number 1 is only visited by 1 individual, door number 2 visited by 2 individuals, door 3 by 2, door 4 by 3, door 5 by 2, door 6 by 3, door 7 by 2, door 8 by 4, door 9 by 3.

Mathematically, we know that prime numbers will be visited by only two individuals, the first individual, and the individual starting at its position. Furthermore, we know that if the door number is a perfect square, in other words, that is an integer and the square of another integer, it has to be visited at least thrice, by the individual with the numbers 1, its own square root, and its own number. Therefore, all perfect squares will have overlapping visits with previous individuals.

Room number	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Number of visitors	1	2	2	3	2	4	2	4	3	4	2	6	2	4	4	5

How does the code adress the problem?:

Screenshots of the results from the RStudio

```
> # printing the first 30 open doors
> print(open_doors)
[1] 1 4 9 16 25 36 49 64 81 100 121 144 169 196 225 256 289 324 361
[20] 400 441 484 529 576 625 676 729 784 841 900
> # printing the first 30 closed doors
> print(closed_doors)
[1] 2 3 5 6 7 8 10 11 12 13 14 15 17 18 19 20 21 22 23 24 26 27 28 29 30
[26] 31 32 33 34 35
```

Problem 6

In this task we are asked to create our own database. We have chosen to create a database for the Education Industry.

Database Idea

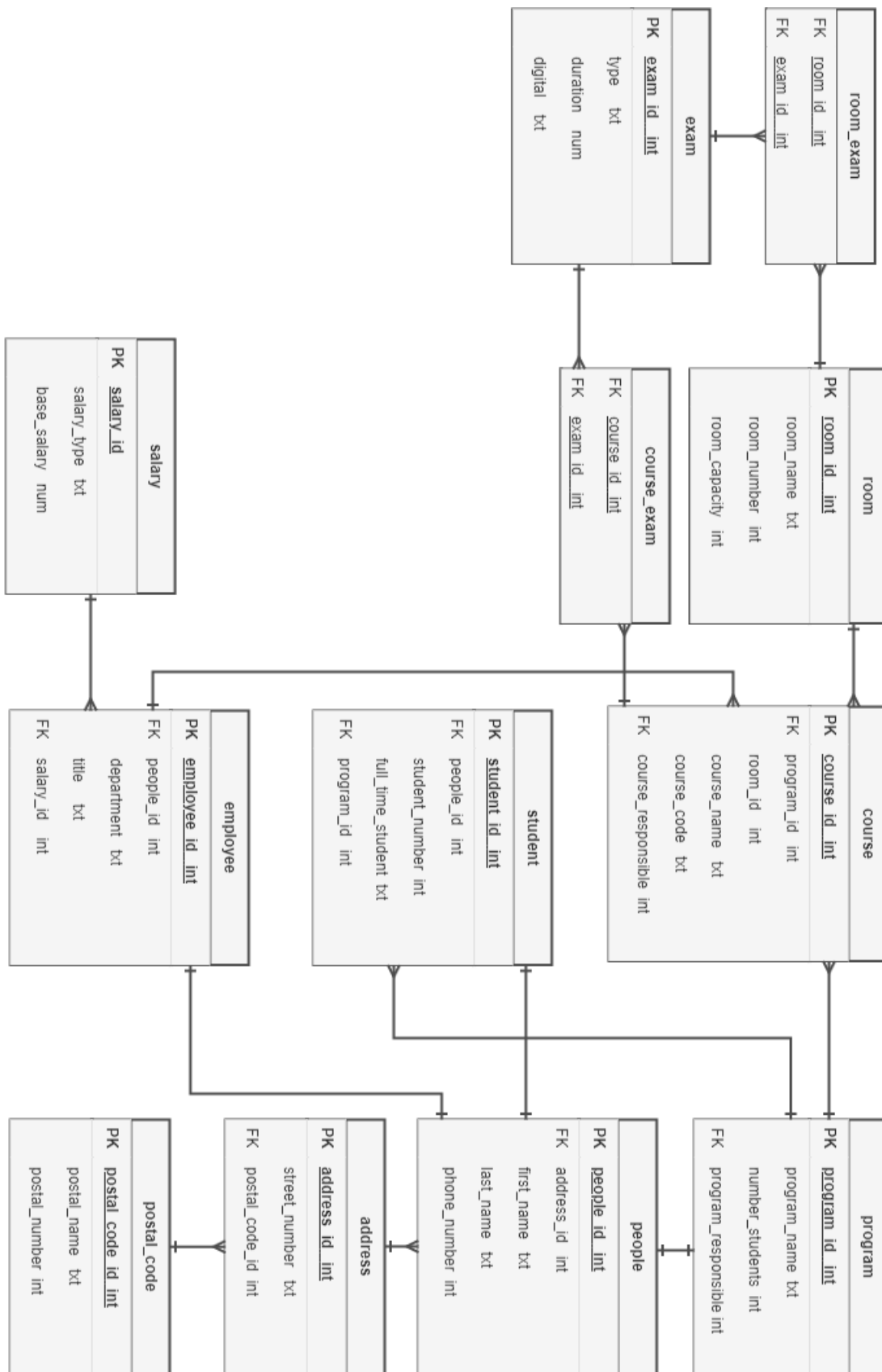
For this problem we have decided to create a database that reflects the Education industry. The idea is to create a database system that enables the user to easily store vital information and that is also easily expendable. The database is made in the context of a higher educational institution in Norway.

The database will reflect the chosen industry because it highlights the importance of a consistent and correct informational system. Furthermore, the database reflects the information that members of an educational institution handle every day - both when it comes to the employees and the students' perspective.

ER-Model

When it comes to the design of the database, we have considered the criteria given, but also the feasibility of the database. We have a total of 12 entities, two one-to-one relationships, four one-to-many relationships and two many-to-many relationships.

The database is illustrated in the entity-relation (ER) diagram on the next page.



Database Creation

SQL Code Documentation

Here we present the SQL Queries that generates the data base. The documentations is provided below the code snippet.

```

1  /* Adding the scemas ----- */
2  /* Adding the address */
3  CREATE TABLE address (
4      address_id  INTEGER NOT NULL,
5      street_number TEXT NOT NULL,
6      postal_code_id  INTEGER NOT NULL,
7      PRIMARY KEY(
8          address_id AUTOINCREMENT
9      )
10 );
11 /* Adding the course table */
12 CREATE TABLE course (
13     course_id INTEGER NOT NULL,
14     program_id INTEGER NOT NULL,
15     room_id INTEGER NOT NULL,
16     course_name TEXT NOT NULL,
17     course_code INTEGER NOT NULL,
18     course_responsible INTEGER NOT NULL,
19     PRIMARY KEY(
20         course_id AUTOINCREMENT
21     )
22     FOREIGN KEY(room_id) REFERENCES room (room_id),
23     FOREIGN KEY(course_responsible) REFERENCES people (people_id),
24     FOREIGN KEY(program_id) REFERENCES program (program_id)
25 );
26 /* Adding the course-exam intermediate table */
27 CREATE TABLE course-exam (
28     course_id INTEGER NOT NULL,
29     exam_id INTEGER NOT NULL,
30     FOREIGN KEY(course_id) REFERENCES course (course_id),
31     FOREIGN KEY(exam_id) REFERENCES exam (exam_id)
32 );
33 /* Adding the employee table */
34 CREATE TABLE employee (
35     people_id INTEGER NOT NULL,
36     department TEXT NOT NULL,
37     title TEXT NOT NULL,
38     salary_id INTEGER NOT NULL,
39     PRIMARY KEY(
40         people_id AUTOINCREMENT
41     )
42 );
43 /* Adding the exam table */
44 CREATE TABLE exam (
45     exam_id INTEGER NOT NULL,
46     type TEXT NOT NULL,
47     duration NUMERIC,

```

```

48     digital TEXT,
49     PRIMARY KEY(
50         exam_id AUTOINCREMENT
51     )
52 );
53 /* Adding the people table */
54 CREATE TABLE people (
55     people_id INTEGER NOT NULL,
56     address_id INTEGER NOT NULL,
57     first_name TEXT NOT NULL,
58     last_name TEXT NOT NULL,
59     phone_number INTEGER NOT NULL,
60     PRIMARY KEY(
61         people_id AUTOINCREMENT
62     )
63 );
64 /* Adding the postal_code table */
65 CREATE TABLE postal_code (
66     postal_code_id INTEGER NOT NULL,
67     postal_name TEXT NOT NULL,
68     postal_number INTEGER NOT NULL,
69     PRIMARY KEY(
70         postal_code_id AUTOINCREMENT
71     )
72 );
73 /* Adding the program table */
74 CREATE TABLE program (
75     program_id INTEGER NOT NULL,
76     program_name TEXT NOT NULL,
77     number_students INTEGER NOT NULL,
78     program_responsible INTEGER NOT NULL,
79     PRIMARY KEY(
80         program_id AUTOINCREMENT
81     )
82 );
83 /* Adding the room table */
84 CREATE TABLE room (
85     room_id INTEGER NOT NULL UNIQUE,
86     room_name TEXT NOT NULL,
87     room_number INTEGER NOT NULL UNIQUE,
88     room_capacity INTEGER NOT NULL,
89     PRIMARY KEY(
90         room_id AUTOINCREMENT
91     )
92 );
93 /* Adding the room_exam intermediate table */
94 CREATE TABLE room_exam (
95     room_id INTEGER NOT NULL,
96     exam_id INTEGER NOT NULL,
97     FOREIGN KEY(room_id) REFERENCES room (room_id),
98     FOREIGN KEY(exam_id) REFERENCES exam (exam_id)
99 );
100 /* Adding the salary table */
101 CREATE TABLE salary (

```



```

102 salary_id INTEGER NOT NULL,
103 salary_type TEXT NOT NULL,
104 base_salary NUMERIC NOT NULL,
105 PRIMARY KEY(
106     salary_id AUTOINCREMENT
107 )
108 );
109 /* Adding the student table */
110 CREATE TABLE student (
111     student_id INTEGER NOT NULL UNIQUE,
112     people_id INTEGER NOT NULL,
113     student_number INTEGER NOT NULL UNIQUE,
114     full_time_student TEXT NOT NULL,
115     program_id INTEGER NOT NULL,
116     PRIMARY KEY(
117         student_id AUTOINCREMENT
118     ),
119     FOREIGN KEY(people_id) REFERENCES people (people_id),
120     FOREIGN KEY(program_id) REFERENCES program (program_id)
121 );

```

Next, we are going to document the different queries, used to create the database. The description follows the same order here, as it does in the .txt file in the submission.

CREATE TABLE address: In this query, we define the attributes as shown in the ER-diagram. The **address_id** is defined to be an integer, and must not be 0, because the the identifier would not work. Here we also add in the **street_number** and **street_number**. The **address_id** is further defined as the primary key, and the primary key is set to **AUTOINCREMENT**, as we do not want the user of the database to input the key-value directly. Next, we have defined the **postal_code_id** to be a foreign key.

CREATE TABLE course: First, we added the attributes from the ER-diagram, and defined the **course_id** to be the primary key, with Autoincrement. Also, we added the foreign keys **room_id** from the **room** table, as well as the **course_responsible** and the **program_id**.

CREATE TABLE course_exam: For this table, we simply create the two foreign keys, being the **course_id** and **exam_id**. Both are defined to be integers.

CREATE TABLE people: For the employee table, we use the attributes from the ER-diagram, and add the **people_id** as the primary key.

CREATE TABLE postal_code: Here we add the attributes from the ER-diagram, and add the **postal_code_id** as the primary key.

CREATE TABLE program: Here we add the attributes from the ER-diagram, and add the **program_id** as the primary key.

CREATE TABLE room: Here we add the attributes from the ER-diagram, and add the **room_id** as the primary key.

CREATE TABLE room_exam: This is the intermediate table in the many-to-many relationship between the room-table and the exam-table. Therefore, we add the two foreign keys in **room_id** and **exam_id**.

CREATE TABLE salary: Here we add the attributes from the ER-diagram, and add the **salary_id** as the primary key.

CREATE TABLE student: Here we add the attributes from the ER-diagram, and add the

`student_id` as the primary key. Also, we add the `people_id` and the `program_id` as foreign keys.

Inserting data to the tables

To add data to the tables, we have used the `INSERT INTO` function. As an example, we take the following query (`student`):

```
INSERT INTO student(people_id, student_number, full_time_student, program_id)
VALUES(1, 155555, "yes", 1),
      (2, 255555, "no", 2),
      (3, 252525, "yes", 3),
      (4, 121212, "yes", 4),
      (8, 123123, "yes", 4);
```

Because the `student`-table's primary key is the `student_id`, and we have made this Autoincrement, we do not need to specify the id when inserting new rows into the tables. This applies to each table, where we have a primary key. Next, we have here inserted the fields `people_id`, which is a foreign key to the `people` table, `student_number`, which is a number assigned to each student when attending the school, `full_time_student`, which is a text indicating if a student is full time or not and lastly a foreign key `program_id` to indicate which program the student is a part of.

Description for each relationship (between tables)

postal_code and address: One Address must and can only have one postal code. One postal code can include multiple addresses, but it is not mandatory.

people and address : One person must have one address, but an address can belong to multiple people. This is a one-to-many relationship.

people and program : One person can be a program responsible, and a program can only have one course responsible. This is a one-to-one relationship.

people and student : One person can be a student, and a student must be a person. This is a one-to-one relationship.

people and employee : One person can be an employee, and an employee must be a person. This is a one-to-one relationship.

program and student : A student must be part of one and only one program, but a program can include multiple students. This is one-to-many relationship.

program and course : A program must include at least one course, but can include many courses. A course must be part of one and only one program. This is a one-to-many relationship.

employee and salary : A employee must have one and only one salary, but many employees can have the same salary. This is a one-to-many relationship.

employee and course : A course can only have one course responsible, but an employee can be course responsible for many courses. This is a one-to-many relationship.

course and room : A course can only have be assigned one room, but a room can host many courses. This is a one-to-many relationship.

room and **exam** : A room can have many exams, and an exam form can be hosted in many rooms. This is many-to-many relationship. We therefore must here introduce an intermediate table, named **room_exam**. One room-exam combination can only be held in one room, but one exam can be hosted in multiple room-exam combinations. That relationship is therefore a one-to-many relationship. On the other side, we have the room-exam to exam. One room-exam combination can only include one exam, but an exam can be hosted in many room-exam combinations. That is a one-to-many relationship.

course and **exam** : A course can have many exams, and an exam form can host many courses. This is many-to-many relationship, so we make an intermediate table, named **course_exam**. One course-exam combination can only host one course, but a course can host multiple room-exam combinations. That relationship is therefore a one-to-many relationship. On the other side, we have the course-exam to exam. One course-exam combination can only include one exam, but an exam can be hosted in many course-exam combinations. That is a one-to-many relationship.

Describing each table

postal_code: The role of this table is to store the postal code and area for every person involved in the institution. Because an address only can have one postal code, but a postal code can have many addresses, we store it in a separate table but a **postal_code_id** as the primary key which we store in the **address** table.

address: For the address table, we have simply stored the addresses for the people involved in the institution. Because multiple people can have the same address, we refer to the address through the primary key **address_id**. The Foreign Key is therefore the **postal_code_id**.

people: The idea of the people table is to store personal data about the different people involved, being either an employee or a student. Because we might want to store the same data about a student and an employee, like the address and name, we therefore gather all this information in the same table, and let the student- or employee-specific information be gathered in the student-table and the employee-table. The primary key in the people table is the **people_id**, and the **address_id** is taken as a foreign key (because multiple people can have the same address).

program: When it comes to the program, we can define different specific program types that the institution should offer. For instance, this could be a "Bachelor in Business and Administration". Such programs are identified by the primary key **program_id**, and a program should have a program responsible, which must be a person (this could also have been an employee, but we want to be able to extract the name of the employee, which is stored in the **people** table).

student: The role of the **student** table is to store information which are relevant regarding a student but not an employee. This could be the student number, or other data as well. The primary key is the **student_id**, and each student must be a person, through the **people_id** foreign key.

employee: The role of the **employee** table is to store information which are relevant regarding an employee but not a student. This could be the salary type, or other data as well. The primary key is the **employee_id**, and each employee must be a person, through the **people_id** foreign key, as the student-table. Also, the salary is a foreign key through the

salary_id.

salary: The role of the salary is to store information on which salary the employees have. The primary key is the **salary_id**.

course: The role of this table is to keep track of all the different courses hosted by the institution. This information would typically be used in the institutions web pages, both in external and internal systems. The primary key is a **course_id**. We also have two foreign keys: a course must be a part of a program, so we have a **program_id** foreign key, and a **course_responsible**, because a course must have a course responsible.

room: On the campus, there are many rooms, which can host classes, exams, or be available for studying. The role of this table is to store information on the different rooms. The primary key is the **room_id**, which identifies a single room. This table has no foreign keys.

exam: The institution also hosts exams. An exam is here defined as a entity which consists of a type (Home exam), duration (3 hours) and a variable indicating if the exam is digital. The point is that multiple courses can use the same exam format. Therefore, we have created the **exam_id**, which is the primary key in the **exam** table, and will serve as a foreign key in the intermediate tables between **exam** and **room**, and **exam** and **course**, which are many-to-many relationships.

room_exam: This table serves as an intermediate table, where each combination of room and exam are stored. In this table we do not have a primary key, but the foreign keys in the **exam_id** and the **room_id**. This creates for a system where one can find information on which rooms the specific types of exams are hosted.

course_exam: Also this table serves as an intermediate table, where we have the foreign keys **exam_id** and the **course_id**. The role of this table is to inform what types of exams a course consists of.

References

- Accelebrate. (n.d.). *Data Science Fundamentals with Python for Healthcare*. <https://www.accelebrate.com/training/data-science-python-healthcare>
- AiCure. (n.d.-a). *About AiCure*. <https://aicare.com/company>
- AiCure. (n.d.-b). AiCure Platform. <https://aicare.com/solutions/aicare-platform>
- Amazon Web Services. (n.d.-a). *AWS HealthLake*. <https://aws.amazon.com/healthlake/>
- Amazon Web Services. (n.d.-b). *What is AWS HealthLake?* <https://docs.aws.amazon.com/healthlake/latest/devguide/what-is-amazon-health-lake.html>
- Asenavi, A. (2022, March). *Python in Healthcare: AI Applications in Hospitals*. DataCamp. <https://www.datacamp.com/blog/python-in-healthcare-ai-applications-in-hospitals>
- Bent, B. (2020, December 23). *Wearablecompute is an open-source python package containing over 50 data and domain-driven features*. DBDP (Digital Biomarker Discovery Pipeline). <https://github.com/brinnaebent/wearablecompute>
- Bronsoler, A., Doyle, J., & Van Reenen, J. (2022). The impact of health information and communication technology on clinical quality, productivity, and workers. *Annual Review of Economics*, 14, 23–46.
- Caruana, R., Lou, Y., Gehrke, J., Koch, P., Sturm, M., & Elhadad, N. (2015). Intelligible models for healthcare: Predicting pneumonia risk and hospital 30-day readmission. *Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining*, 1721–1730.
- Eastwood, B. (2023). How to navigate structured and unstructured data as a healthcare organization. *HealthTech*. <https://healthtechmagazine.net/article/2023/05/structured-vs-unstructured-data-in-healthcare-perfcon>
- Florice, C., Epifano, J., Caamano, S., Kark, S., Christie, R., Masino, A., & Paredes, A. D. (2022). Opening Access to Visual Exploration of Audiovisual Digital Biomarkers: an OpenDBM Analytics Tool. *arXiv preprint arXiv:2210.01618*.
- Horowitz, B. (2022). The Current State of AI in Healthcare and Where It’s Going in 2023. *HealthTech*. <https://healthtechmagazine.net/article/2022/12/ai-healthcare-2023-ml-nlp-more-perfcon>
- Jordan, S., Fontaine, C., & Hendricks-Sturup, R. (2022). Selecting privacy-enhancing technologies for managing health data use. *Frontiers in Public Health*, 10, 814163.
- Karthan, M., Martin, R., Holl, F., Swoboda, W., Kestler, H. A., Pryss, R., & Schobel, J. (2022). Enhancing mhealth data collection applications with sensing capabilities. *Frontiers in Public Health*, 10, 926234.
- Keniya, R., Khakharia, A., Shah, V., Gada, V., Manjalkar, R., Thaker, T., & Mehendale, N. (2020). Disease prediction from various symptoms using machine learning [Available at SSRN 3661426].
- Kumar, Y., Koul, A., Singla, R., & Ijaz, M. F. (2022). Artificial intelligence in disease diagnosis: A systematic literature review, synthesizing framework and future research agenda. *Journal of ambient intelligence and humanized computing*, 1–28.
- Miller, A. R., & Tucker, C. (2009). Privacy protection and technology diffusion: The case of electronic medical records. *Management science*, 55(7), 1077–1093.

- Mohanty, S., Mishra, A., & Saxena, A. (2021). Medical data analysis using machine learning with KNN, 473–485.
- Murdoch, B. (2021). Privacy and artificial intelligence: Challenges for protecting health information in a new era. *BMC Medical Ethics*, 22(1), 1–5.
- Oscarson, N., Taylor, B., & Bentley, M. (2023, June). *Qventus Solutions 2023: Achieving Operational Improvements through AI, Machine Learning, and Behavioral Science*. KLAS Research.
- Paige. (n.d.-a). *Empowering decisions through data*. <https://paige.ai/about/>
- Paige. (n.d.-b). *Reliable AI for Cancer Detection Diagnosis*. <https://paige.ai/diagnostic-ai/>
- Qventus. (n.d.-a). *About Qventus*. <https://qventus.com/about/>
- Qventus. (n.d.-b). *Perioperative solution*. <https://qventus.com/perioperative/>
- Qventus. (n.d.-c). *Qventus for inpatient units*. <https://qventus.com/inpatient/>
- Salazar-Reyna, R., Gonzalez-Aleu, F., Granda-Gutierrez, E. M., Diaz-Ramirez, J., Garza-Reyes, J. A., & Kumar, A. (2022). A systematic literature review of data science, data analytics and machine learning applied to healthcare engineering systems. *Management Decision*, 60(2), 300–319.
- Shah, T., O’Neal, M., Goel, A., & Grant, T. (2023). Reinventing care delivery is essential to solve the clinician shortage. <https://www.accenture.com/content/dam/accenture/final/accenture-com/document/Accenture-Reinventing-Care-1.pdf>
- Stobierski, T. (2021, February 2). *8 steps in the data life cycle*. Harvard Business School Online. <https://online.hbs.edu/blog/post/data-life-cycle>
- University of Pittsburgh. (2021, January 13). *The role of data analytics in health care*. <https://online.shrs.pitt.edu/blog/data-analytics-in-health-care/>
- Widodo, C., Adi, K., & Gernowo, R. (2020). Medical image processing using python and open cv. *1524*(1), 012003.
- Yuan, S., Chan, H. S., & Hu, Z. (2017). Using PyMOL as a platform for computational drug design. *Wiley Interdisciplinary Reviews: Computational Molecular Science*, 7(2), e1298.
- Zhao, Y., Qiao, Z., Xiao, C., Glass, L., & Sun, J. (2021). Pyhealth: A python library for health predictive models. *arXiv preprint arXiv:2101.04209*.